08-Decorator Pattern

The **Decorator Pattern** is a design pattern that lets you **add new behavior to an object without changing its original code**.

យើងប្រើវា ពេលដែលយើងចង់ដាក់បន្ថែម មុនងាច្នៃចូលវា។

🧠 What is the Decorator Pattern (simple explanation)

👉 The **Decorator Pattern** is a design pattern that lets you **add new behavior to an object without changing its original code**.

Instead of editing the original class, you:

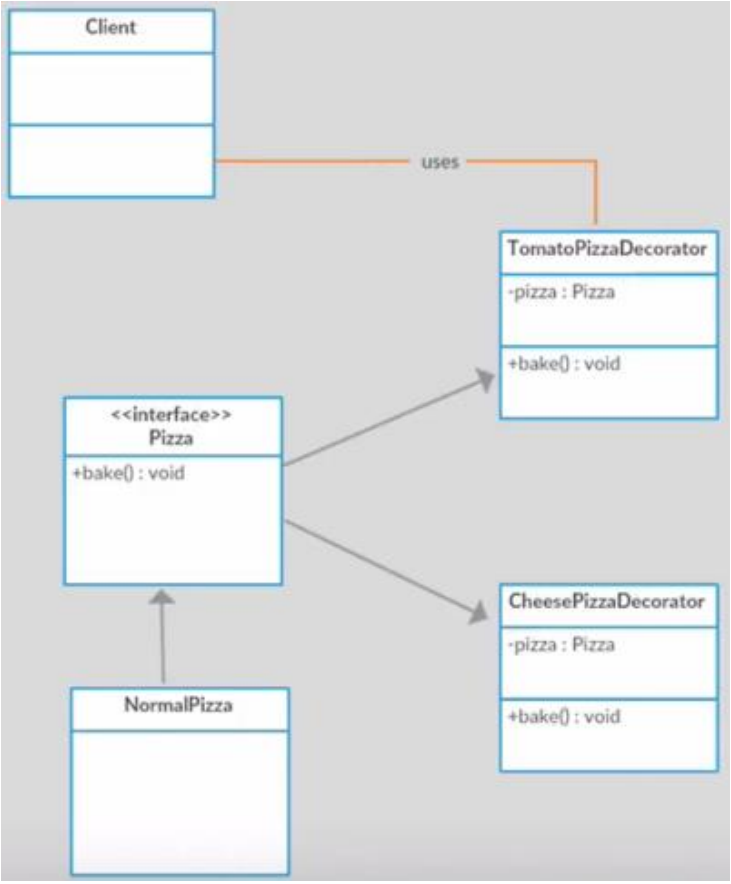
- **wrap it**
- and **add extra features on top**

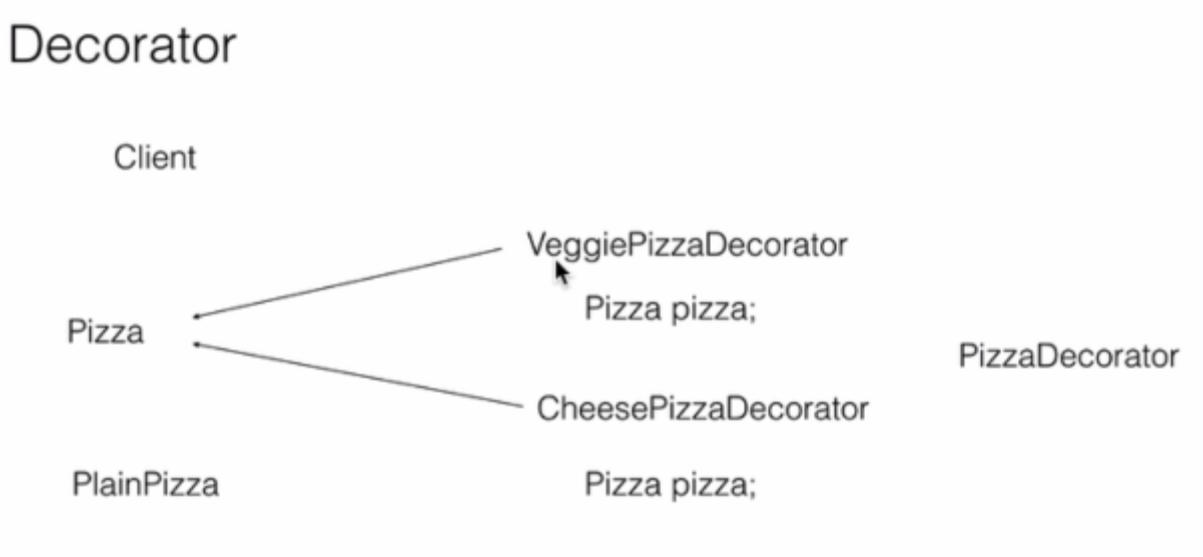
🍕 Easy Idea (Pizza Example)

Think of a basic pizza:

- You start with **plain pizza**
- Then you add:
  - veggies 🥦
  - cheese 🧀
  - mushrooms 🍄

👉 You are **not changing the original pizza**, you are just **adding layers**





🎯 Why Use Decorator Pattern?

✅ Advantages

- No need to modify existing code
- Flexible (combine features easily)
- Avoid too many subclasses

❌ Without Decorator (Bad way)

You would need many classes like:

- `SalaryWithBonus`
- `SalaryWithTax`
- `SalaryWithBonusAndTax`
- `SalaryWithBonusTaxInsurance`

👉 This becomes messy 😵

🔁 One Simple Sentence

👉 Decorator Pattern = **Wrap an object and add new behavior dynamically**

If you want, I can:

- draw a **diagram (very clear visually)**
- or convert your **pizza example into a full runnable project**

Example: PizzaWrapper ការវិចខ្ចប់ Pizza

1- Interface : where we put the function

udemy-course-design-pattern ▸ src/main/java ▸ com.vibol.corepattern.decorator08 ▸ Pizza ▸

```java
package com.vibol.corepattern.decorator08;

public interface Pizza {

    public void bake();

}

```

2- Concrete class from Interface

udemy-course-design-pattern [boot] [Udemy-Design-Pattern Section10
src/main/java
com.vibol.corepattern.AbstractFactory03
com.vibol.corepattern.adapter05
com.vibol.corepattern.adapter05.hw01
com.vibol.corepattern.adapter05.hw02
com.vibol.corepattern.command07
com.vibol.corepattern.decorator08
CheesePizzaDecorator.java
Pizza.java
PizzaDecorator.java
PizzaShop.java
PlainPizza.java
VeggiePizzaDecorator.java
com.vibol.corepattern.decorator08.hw
BaseSalary.java

udemy-course-design-pattern ▸ src/main/java ▸ com.vibol.corepattern.decorator08 ▸ PlainPizza ▸

```java
package com.vibol.corepattern.decorator08;

public class PlainPizza implements Pizza {

    // this class like the plain nothing on it
    // this is the concrete component that implement from pizza

    @Override
    public void bake() {
        System.out.println("Baking Plain Pizza");
    }

```

3- Decorator class (យើងគិតវាដូចជាប្រអប់មួយចឹង ដែលយើងអាចដាក់ចូលបានដើម្បីបង្កើតជា Cheese Pizza or Vegi Pizza)

VeggiePizzaDecorator.java | PizzaDecorator.java

udemy-course-design-pattern ▸ src/main/java ▸ com.vibol.corepattern.decorator08 ▸ PizzaDecorator ▸

```java
package com.vibol.corepattern.decorator08;

public class PizzaDecorator implements Pizza {

    //It stores another Pizza object inside : This is called wrapping
    private Pizza pizza;


    //Now this decorator is wrapping that pizza
    public PizzaDecorator(Pizza pizza) {
        this.pizza = pizza;
    }

    @Override
    public void bake() {
        pizza.bake();
    }


    /*
    Simple Analogy

    Think of this class like a transparent box:

    You put a pizza inside
    The box itself does nothing
    But later, you can create special boxes:
    Veggie box → adds toppings
    Cheese box → adds cheese

    */
}
```

4- VeggiePizzaDecorator

```java
package com.vibol.corepattern.decorator08;

public class VeggiePizzaDecorator extends PizzaDecorator {

    // contractor then it called the base pizza first
    public VeggiePizzaDecorator(Pizza pizza) {
        super(pizza);
    }

    public void bake() {
        super.bake();
        System.out.println("Adding veggie toppings");
    }
}
```

សម្រាប់ class VeggiePizzaDecorator នឹងវាធ្វើការ បង្កើត overight method bake();

5- CheesePizzaDecorator

VeggiePizzaDecorator.java | CheesePizzaDecorator.java

udemy-course-design-pattern ▸ src/main/java ▸ com.vibol.corepattern.decorator08 ▸ CheesePizzaDecorator ▸

```java
package com.vibol.corepattern.decorator08;

public class CheesePizzaDecorator extends PizzaDecorator {

    // contractor then it called the base pizza first
    public CheesePizzaDecorator(Pizza pizza) {
        super(pizza);
    }

    public void bake() {
        super.bake();
        System.out.println("Adding Cheese toppings");
    }
}
```

សម្រាប់ class CheesePizzaDecorator នឹងវាធ្វើការ បង្កើត overight method bake();

6- Testing: យើងគ្រាន់តែ ដាក់ wrapper ពីលើវា បន្ថែមទៀត

PizzaShop.java

udemy-course-design-pattern ▸ src/main/java ▸ com.vibol.corepattern.decorator08 ▸ PizzaShop ▸

```java
package com.vibol.corepattern.decorator08;

public class PizzaShop {

    public static void main(String[] args) {
//      Pizza pizza = new PlainPizza();
//      Pizza pizza = new CheesePizzaDecorator(new PlainPizza());

        Pizza pizza = new VeggiePizzaDecorator(new CheesePizzaDecorator(new PlainPizza()));

        pizza.bake();
    }

}
```

Problems | Javadoc | Declaration | Terminal | Console | Boot Dashboard

<terminated> PizzaShop [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe (Apr 24, 2026, 9:42:06 AM – 9:42:07 AM) [pid: 7684]

```
Baking Plain Pizza
Adding Cheese toppings
Adding veggie toppings
```